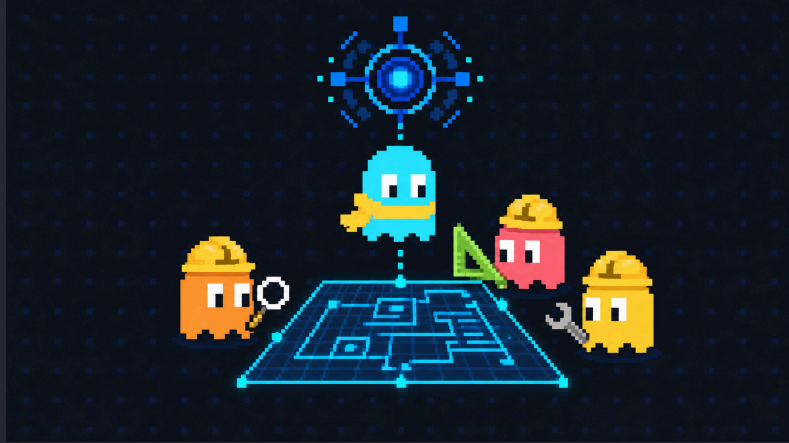




EASY AI - GLOSSARY EDITION

# AI 코딩 용어사전

하네스부터 멀티에이전트, RAG, LLM 위키, 루프 엔지니어링까지 — 강의 1·2·3편의 핵심 용어를 선별해, 비유 하나와 그림 하나로 끝내는 책자.

8개 장

그림 29점

2026-08 기준

[🔗 웹으로 보기](https://kimjiwon-slide.vercel.app/easyai) `kimjiwon-slide.vercel.app/easyai`

# 1 기본기 — LLM이라는 엔진

결국 LLM으로 통한다.

## LLM Large Language Model

다음에 올 말을 확률로 예측하도록 거대한 텍스트로 학습된 모델.

☞ **쉽게 말하면** — 세상의 글을 엄청나게 읽고 **다음에 올 단어를 맞히는 연습**만 반복한 존재. 학습을 많이 해서 다음 단어를 더 잘 맞힐수록 좋은 모델입니다. 정답을 아는 게 아니라 **가장 그럴듯한 걸** 말합니다.

## 토큰 · 컨텍스트 윈도우 token · context window

토큰 = 모델이 글을 세는 단위(한글 1자 ≈ 1~2토큰). 컨텍스트 윈도우 = 모델이 **한 번에 볼 수 있는 토큰 한도**.<sup>\*2</sup>

☞ **쉽게 말하면** — 컨텍스트 윈도우는 **책상 크기**입니다. 책상이 아무리 커도, 서류를 산처럼 쌓으면 정작 필요한 종이를 못 찾습니다.



⊗ **흔한 오해** — "1M 토큰 모델이니깐 1M을 다 넣으면 되지."

☑ **사실** — **스펙에 적힌 최대치**(1M)와 **실효 컨텍스트**(정확도가 유지되는 구간, 대략 200~400K ●)는 다릅니다. 200K 모델이 5만 토큰에서 이미 정확도를 잃는 연구도 있습니다(Chroma, 18개 모델 ●)<sup>\*6</sup>.

## 컨텍스트 윈도우 사용량

전체 100% 중 지금 요청이 차지한 공간을 비율로 봅니다



● 시스템 지침 18%

● 도구 결과 14%

● 검색 문서 16%

● 이전 대화 12%

● 현재 질문 8%

● 여유 32%

가득 채우는 것이 목표가 아니라, 남은 자리까지 함께 보는 것이 핵심입니다

그림 1 · 전체 100%를 항목별로 보기 — 무엇이 공간을 차지하는지도 함께 확인합니다

## 추론 모델 · 생각 토큰 reasoning model · thinking budget



답하기 전에 "생각 토큰"을 태우며 문제를 풀어보는 모델. 수학·코드·계획에 강하지만 느리고 비쌉니다. 생각의 양을 조절하는 손잡이가 thinking budget(effort).

☞ **쉽게 말하면** — 일반 모델은 즉답하는 사람, 추론 모델은 **연습장에 풀어보고 답하는 사람**. 연습장 쓰는 시간만큼 느리고, 연습장 종이값(토큰)이 듅니다.

### 질문에서 답까지, 생각 토큰이 쓰이는 순서



그림 2 · 생각 토큰이 쓰이는 자리 — 답하기 전에 풀이와 검토가 한 단계 더 있습니다

## 할루시네이션(환각) · 그라운드링 hallucination · grounding



할루시네이션(환각) = 근거 없는 내용을 그럴듯하게 생성하는 것. 그라운드링 = 답을 실제 근거(문서·코드·검색결과)에 묶는 것.

☕ **쉽게 말하면** — LLM은 "모른다"보다 "그럴듯한 오답"을 잘 만듭니다. 그래서 5장(RAG·검색)과 7장(하네스의 검증 루프)이 존재합니다. **이 책 절반은 사실 환각 대책입니다.**

**그라운드링은 더 쉽게 말하면 — 답하기 전에 힌트를 최대한 많이 줘야 하는 것입니다.**<sup>\*1</sup> 관련 문서·코드·검색 결과를 미리 옆에 놓아 주면, 모델이 스스로 지어낼 여지가 그만큼 줄어듭니다.



### 출처를 다시 묻는 습관

**확인 없이 넘어가면**

"이미 지원됩니다"

**출처를 못 댁니다**

**출처를 물으면**

"이 문서 3쪽에 있습니다"

**바로 확인됩니다**

**답이 맞는지가 아니라, 어디서 나왔는지를 묻는 것이 핵심입니다**

그림 3 · “없는 기능인데도 됩니다”를 의심하기 — 답마다 어디서 나왔는지를 다시 확인합니다

## 오픈웨이트 vs 폐쇄형 open-weight vs closed

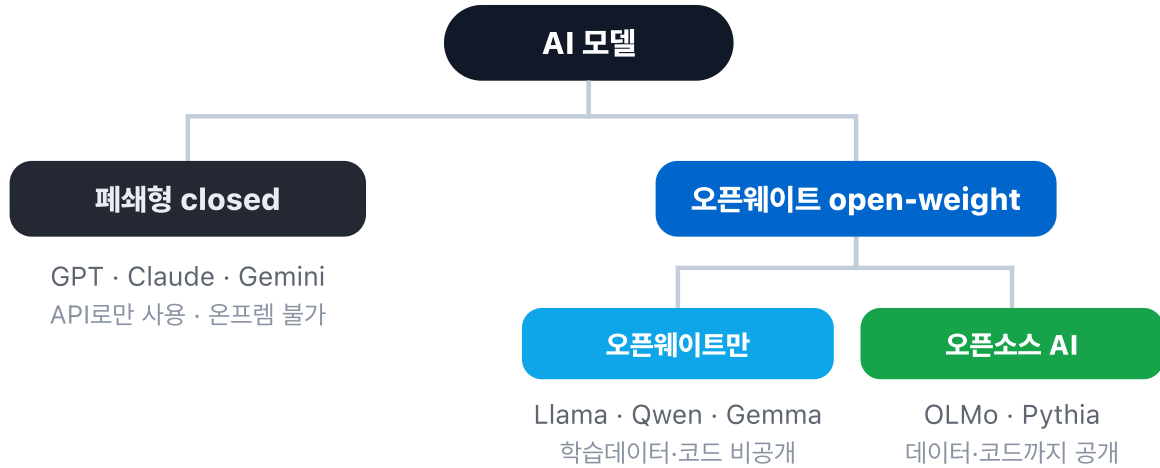


가중치 파일을 내려받아 **직접 운영하는 서버에서** 돌릴 수 있으면 오픈웨이트, API로만 쓸 수 있으면 폐쇄형. 오픈웨이트 ≠



오픈소스(학습 데이터·코드 공개와는 다른 얘기).

☞ **쉽게 말하면 — 온프레임(내부망) 가능 여부**가 여기서 갈립니다. 금융권처럼 데이터가 밖으로 나갈 수 없는 곳에서는 이 축이 모델 선택의 첫 번째 기준이 됩니다.



오픈소스 AI는 대부분 오픈웨이트 · 그러나 오픈웨이트가 곧 오픈소스는 아닙니다

그림 4 · 폐쇄형 → 오픈웨이트 → (그중 일부만) 오픈소스 AI

⊗ **흔한 오해** — "가중치를 받아서 내부망 서버에서 돌릴 수 있으니 오픈소스네요."

☑ **사실** — 아닙니다. Llama·Qwen은 **가중치 공개·자체 서버 실행까지는 가능**하지만 학습 데이터 전체와 학습 코드·파이프라인은 공개하지 않습니다. 그래서 오픈웨이트라고 부릅니다. 반대로 코드만 공개하고 모델 파일을 주지 않으면 실제로 쓸 수 없기 때문에, **오픈소스 AI는 대부분 오픈웨이트를 겸합니다.** ●

**파인튜닝(Fine-tuning)**은 모델의 가중치(weights)를 다시 학습해 모델 자체를 바꾸는 방법입니다. 반면 **RAG**는 가중치를 건드리지 않고 검색한 자료를 이번 요청의 **컨텍스트**에 함께 넣어 읽게 합니다. **지식화**는 문서를 파싱·구조화해 검색 가능한 형태로 준비하는 과정입니다.

⊗ **흔한 오해** — "문서를 지식화하면 모델이 학습되나요?"

✔ **사실** — 아닙니다. 지식화와 RAG에서는 **가중치가 전혀 바뀌지 않습니다**. 질문이 들어올 때마다 필요한 문서를 검색해 현재 요청의 컨텍스트에 일시적으로 추가할 뿐입니다. 파인튜닝처럼 모델을 다시 학습하는 과정이 아닙니다. ●

☞ **가중치(Weights)란?** — LLM이 학습으로 얻은 수많은 숫자(파라미터)입니다. 문법·프로그래밍·언어 패턴 같은 능력을 이루는 모델 내부의 장기 기억에 해당합니다. 파인튜닝은 이 숫자를 조금 바꾸지만, RAG·지식화는 끝까지 그대로 둡니다.

### 세 흐름을 구분하면

- **Fine-tuning** — 문서 → 학습 → 가중치 변경 → 새로운 모델. 학습이 끝나면 원문을 매번 붙이지 않아도 학습한 패턴을 사용할 수 있지만 비용과 시간이 큼니다.
- **RAG** — 질문 → 검색 → 관련 문서 추출 → 컨텍스트에 추가 → LLM 답변. 모델 가중치는 바뀌지 않습니다.
- **지식화** — 문서 → 파싱·구조화 → 검색 저장소에 보관. 임베딩·벡터DB를 쓸 수도 있고, 파일·키워드 검색을 쓸 수도 있습니다. RAG를 위한 준비 작업인 경우가 많습니다.

☞ **쉽게 말하면** — **RAG의 본질은 "검색해서 이번 요청의 컨텍스트에 추가하는 것"**입니다. 컨텍스트 원도우가 모델이 한 번에 읽을 수 있는 작업 책상의 크기라면, 프롬프트는 그 책상 위에 실제로 올려놓은 종이들입니다. 시스템 지침, 검색 문서, 도구 결과, 이전 대화, 사용자의 질문이 하나의 입력을 이루고 모델은 그 전체를 읽어 답합니다.

이 전체가 프롬프트



그림 5 · 사용자가 친 문장만 프롬프트가 아닙니다 — 이 상자 안 전체가 프롬프트

#### 외우는 공식 세 줄

- **가중치** = 학습으로 바뀌는 모델 내부의 수많은 숫자(파라미터). 파인튜닝은 가중치를 바꿉니다.
- **컨텍스트 윈도우** = LLM이 한 번에 볼 수 있는 최대 입력 공간입니다. 예를 들어 128K면 최대 12만 8천 토큰까지 읽습니다.
- **프롬프트** = 그 공간에 실제로 들어가는 전체 입력입니다. 시스템 지침·검색 결과·도구 결과·이전 대화·사용자 질문이 모두 포함됩니다.
- **RAG·지식화** = 지식화한 문서를 검색해 컨텍스트에 추가하는 방식입니다. 지식화한 파일을 시스템 프롬프트 뒤에 붙이는 6장의 LLM 위키도 같은 구조입니다.<sup>\*4</sup>

#### 한 문장으로

"RAG는 검색 → 컨텍스트에 추가 → 함께 읽고 추론하는 방식입니다." 문서를 넣었다고 모델이 학습되는 것은 아니며, 가중치는 끝까지 그대로입니다.

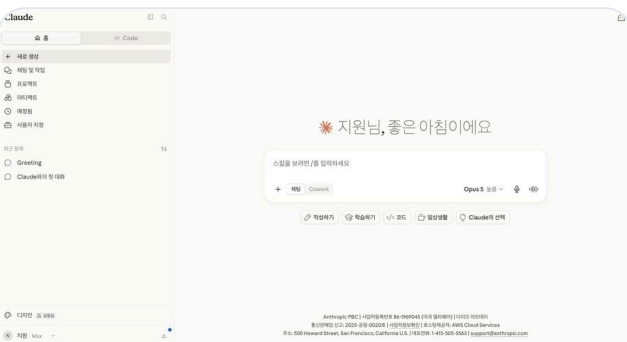
## 2 코드 어시스턴트 — 에이전트의 탄생

"채팅으로 물어보기"에서 "일을 맡기기"로.

### LLM을 만나는 창구 interfaces

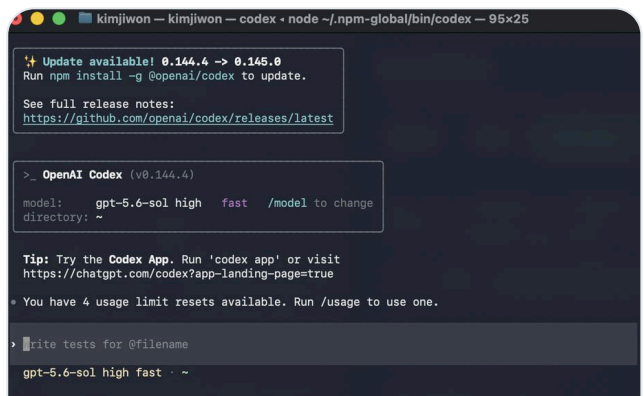
처음에는 대부분 ChatGPT처럼 웹 화면에서 질문하고 답을 받았습니다. 지금은 LLM을 사용하는 창구가 훨씬 다양해졌습니다.

- 웹 UI — ChatGPT·Claude처럼 질문하고 답을 받는 창구
- CLI — Claude Code·Codex처럼 터미널에서 작업하는 창구
- IDE — Cursor·VS Code 확장처럼 개발 환경에 붙는 창구
- 데스크톱 앱 · 클라우드 에이전트 · PR 리뷰 봇 — 실행 환경과 역할이 더해진 창구



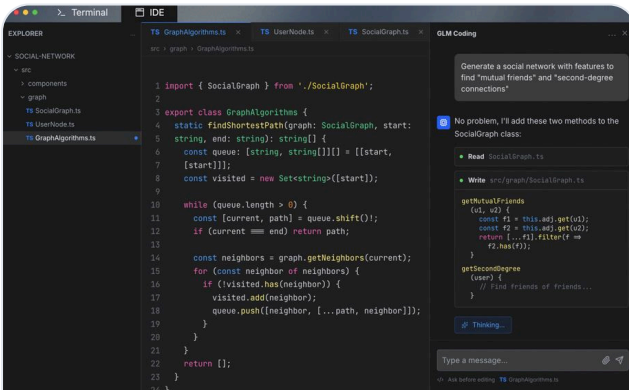
#### 웹 UI

ChatGPT·Claude처럼 질문하고 답을 받는 창구



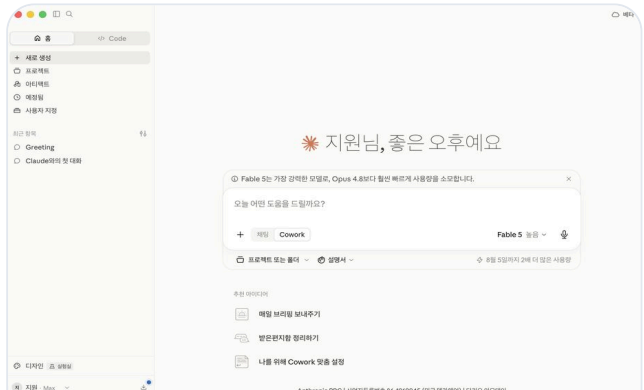
#### CLI

Claude Code·Codex처럼 터미널에서 작업하는 창구



#### IDE 확장

Cursor·VS Code 확장처럼 개발 환경에 붙는 창구



#### 데스크톱 앱

실행 환경과 역할이 더해진 창구

핵심은 창구의 개수가 아니라, 각 창구가 모델에 줄 수 있는 **권한과 작업 범위**가 다르다는 점입니다. 웹에서는 주로 질문과 답변을 주고받지만, CLI·IDE·데스크톱 환경에서는 파일을 읽고 코드를 수정하고 명령을 실행하고 Git을 다루는 등 실제 작업을 수행할 수 있습니다. 그래서 AI는 단순한 **채팅 모델**에서 **업무를 수행하는 에이전트**로 발전했습니다.

## 에이전틱 루프 agentic loop



에이전틱 루프는 모델이 목표를 향해 **계획하고** → **도구를 실행하고** → **결과를 확인하고** → **다음 행동을 판단하는** 과정을 반복하는 구조입니다. 작업이 끝날 때까지 이 순환을 이어 갑니다.

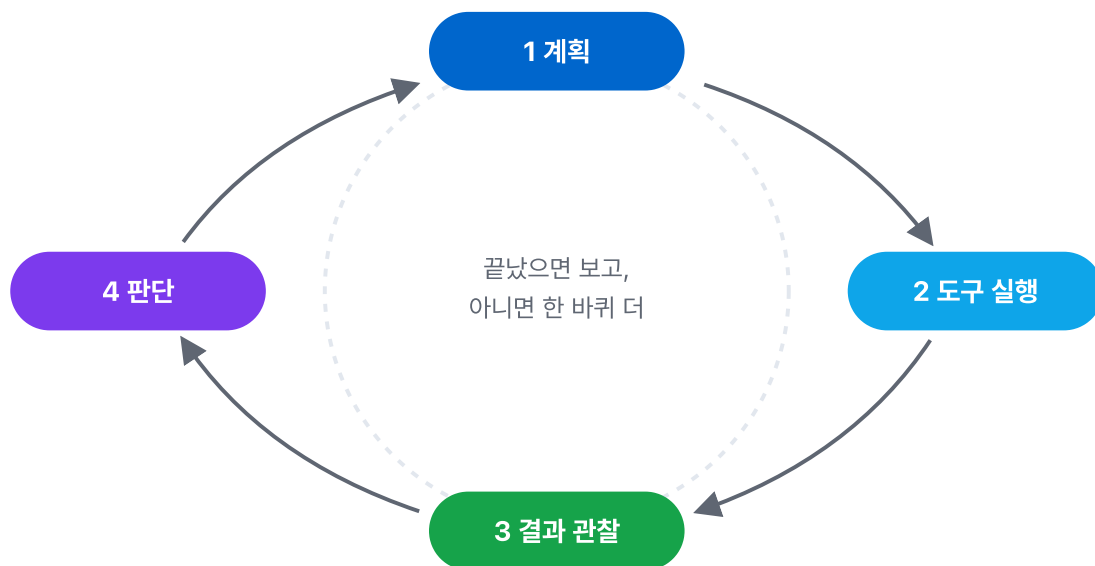


그림 6 · 에이전틱 루프

### 핵심 문장

모델은 "무엇을 하겠다"고 **제안할 뿐**이고, 실제로 할지 말지는 도구·권한·검증 절차가 정합니다. 그래서 보안은 모델이 아니라 그 바깥에 겁니다(7장 하네스).

## Workflow vs Agent 워크플로 vs 에이전트



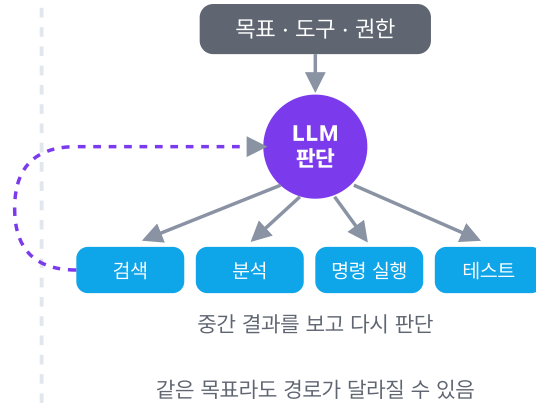
**워크플로**는 사람이 단계와 순서를 미리 조립하고, **에이전트**는 목표와 허용된 도구·권한 안에서 실행 중 다음 행동을 선택합니다.

|       | Workflow      | Agent           |
|-------|---------------|-----------------|
| 작업 경로 | 사람이 미리 조립     | 모델이 실행 중 선택     |
| 다음 행동 | 정해진 단계·조건     | 목표와 중간 결과       |
| 결과    | 비교적 일정·예측 가능  | 경로·결과가 달라질 수 있음 |
| 장점    | 안정성·재현성·감사 용이 | 유연성·탐색·예외 대응    |
| 적합한 일 | 반복·정형·규정 준수   | 탐색·분석·낯선 문제     |

### Workflow — 조립된 길을 따라감



### Agent — 실행 중에 길을 고름



워크플로는 정해진 길을 실행하고, 에이전트는 길을 고르면서 실행합니다  
그만큼 유연하지만 예측 가능성은 낮아집니다

그림 7 · 경로가 고정되어 있으나, 실행 중에 선택되느냐

☕ **쉽게 말하면** — 워크플로는 정해진 길을 실행하고, 에이전트는 허용된 범위에서 길을 고르며 실행합니다. 유연하지만 예측 가능성은 낮아집니다.

⊗ **흔한 오해** — 에이전트가 완전히 자유롭게 행동한다.

☑ **사실** — 사람이 허용한 도구·권한·규칙 안에서 움직이는 **제한된 자율성**입니다.

## AGENTS.md · CLAUDE.md 프로젝트 지침 파일



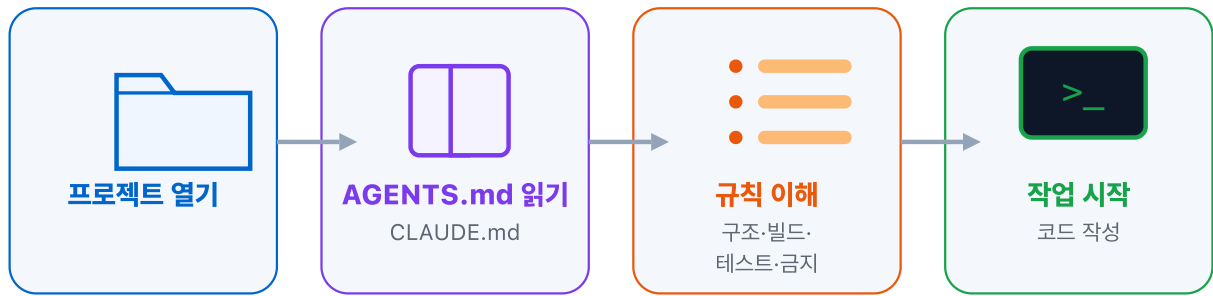
에이전트가 프로젝트를 열 때 먼저 읽는 **온보딩 문서**입니다. 매번 프롬프트로 설명하지 않아도 규칙과 작업 방법을 참고합니다.

- 코딩 스타일·폴더 구조
- 빌드·실행·테스트 방법
- 사용 라이브러리와 수정하면 안 되는 파일
- Git 규칙과 프로젝트 특수 규칙

**대표 파일** — `AGENTS.md` 는 여러 AI 도구가 함께 쓰는 표준 지침, `CLAUDE.md` 는 Claude Code용 지침입니다.  
`.techai.md` · `rules.md` 처럼 이름이 달라도 목적은 거의 같습니다.

☕ **쉽게 말하면** — AI 에이전트에게 주는 프로젝트 사용설명서입니다. 에이전트는 **매 세션이 첫 출근**이라 이 문서를 읽고 일을 시작합니다. 신입사원에게 주는 온보딩 문서와 같아서, 잘 써 둘수록 매일이 좋아집니다.

## 에이전트의 첫 출근



공통 규칙을 파일로 남기면 매번 다시 설명할 일이 줄어듭니다

그림 8 · 열기 → 읽기 → 이해 → 작업 — 공통 규칙을 파일로 남기면 매번 다시 설명할 일이 줄어듭니다

## 바이브 코딩 → 에이전틱 엔지니어링 vibe coding → agentic engineering



바이브 코딩 = 결과만 보고 코드는 안 읽으며 "느낌으로" 만들기. 프로토타입엔 좋지만 운영엔 위험. 2026년의 방향은 **에이전틱 엔지니어링** — 에이전트에게 맡기되 스펙·검증·리뷰의 공학 규율을 유지하는 것 ●.

☕ **쉽게 말하면** — 바이브 코딩이 "맛만 보고 합격"이라면, 에이전틱 엔지니어링은 **위생검사까지 통과** 시키는 것. 함께 쓰는 방법: 코드는 건드리지 않고 **계획만 먼저 세우게 하기**(Plan mode), **테스트를 먼저 주고 통과할 때까지 반복**시키기, **무엇을 만들고 언제 끝인지 먼저 적어 두기**.

## 말로 끝내는가, 결과까지 확인하는가

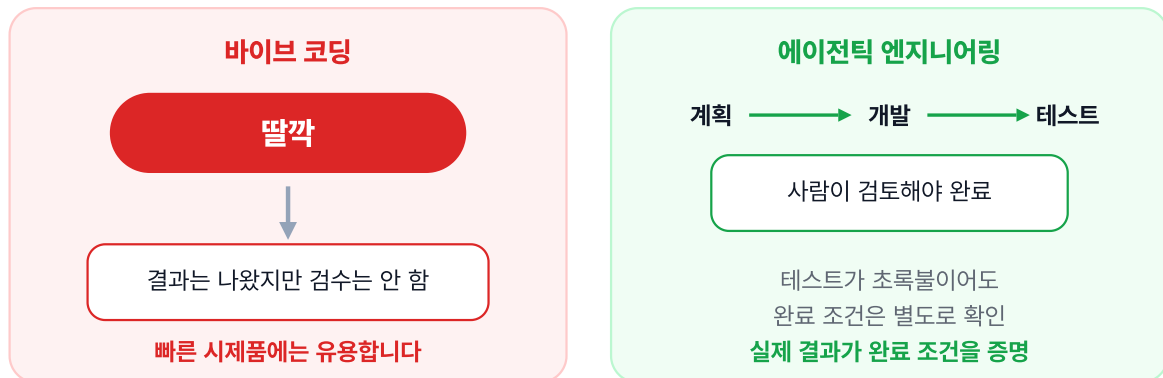
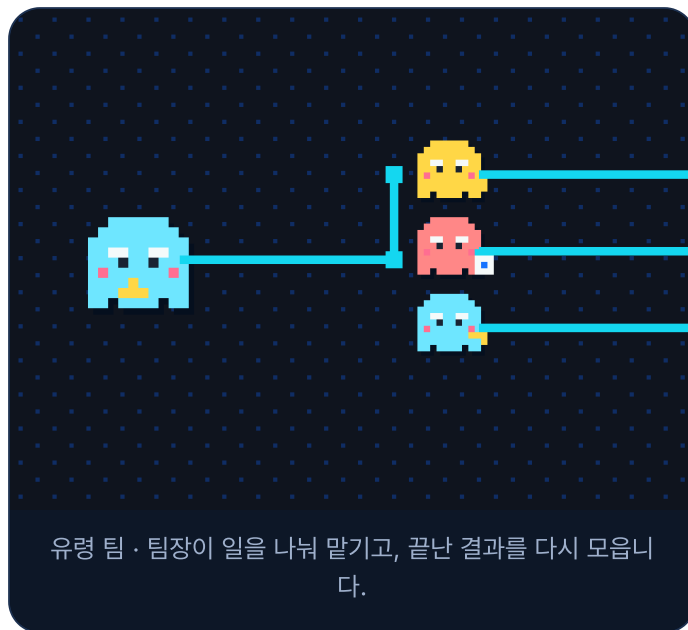


그림 9 · 딸깍으로 끝내지 않습니다 — 실제 결과가 완료 조건을 통과해야 끝입니다

### 3 멀티에이전트 — 혼자보다 팀, 단 공짜는 아님

보통 팀장 역할의 모델 하나가 일을 나눠 맡기고, 팀원 모델 여러 개가 각자 맡은 부분만 처리한 뒤 결과를 올려보냅니다. 빨라지는 대신 쓰는 돈과 토큰은 에이전트 수만큼 배로 듭니다.

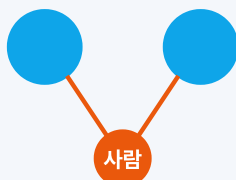


#### 멀티에이전트를 운영하는 3가지 방식

사람이 관리 · 팀장이 관리 · 팀끼리 대화

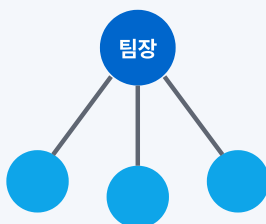


##### ① 사람이 관리



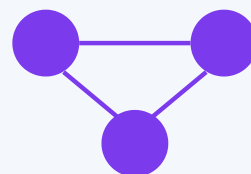
창을 여러 개 열고 사람이 조율

##### ② 팀장이 관리



팀장 에이전트가 나눠 맡김

##### ③ 팀끼리 대화



팀원끼리 직접 주고받음

그림 10 · 뒤로 갈수록 사람이 덜 개입합니다. 좋아지는 순서가 아니라 통제 방식의 차이입니다 ●



| 용어      | 쉽게 말하면                                        |
|---------|-----------------------------------------------|
| 오케스트레이터 | 일을 나눠 맡기고 결과를 합치는 <b>팀장 에이전트</b>              |
| 서브에이전트  | 맡은 역할만 하는 <b>팀원 에이전트</b> . 자기 작업에 필요한 것만 받습니다 |
| 에이전트 팀  | 팀장을 거치지 않고 <b>팀원끼리 직접 주고받는 구조</b>             |
| 오케스트레이션 | 누가 언제 어떤 순서로 일할지 <b>관리하는 것</b>                |

🔊 **쉽게 말하면** — 셋은 좋아지는 순서가 아니라 **사람이 얼마나 개입하느냐의 차이**입니다. 실무에서 가장 다루기 쉬운 형태는 **팀장이 일을 나누고, 팀원은 각자 격리된 공간에서 일하고, 전체 대화가 아니라 요약만 돌려주는 구조**입니다 🟡. 요약만 올려보내는 이유는, 팀원이 읽은 것을 통째로 올리면 팀장의 컨텍스트가 금방 오염되기 때문입니다.

## 자주 쓰는 4가지 방식 순서대로 · 동시에 · 팀장이 나눠서 · 넘기기



| 패턴        | 구조                   | 언제                                 |
|-----------|----------------------|------------------------------------|
| 순서대로      | A → B → C로 결과를 넘김    | 앞이 끝나야 뒤를 할 수 있을 때 (구현 → 테스트 → 리뷰) |
| 동시에       | 여럿이 같이 하고 마지막에 합침    | 서로 기다릴 필요가 없을 때 (조사·비교·여러 관점 리뷰)   |
| 팀장이 나눠서   | 오케스트레이터가 쪼개고 맡기고 합침  | 큰 기능을 영역별로 나눌 때                    |
| 담당자에게 넘기기 | 주도권 자체를 다른 에이전트에게 넘김 | 질문 종류에 따라 담당자가 갈릴 때                |

여기에 **여러 에이전트가 각자 답을 내고 서로 채점하게 하는** 방식을 더하기도 합니다. 다만 채점을 맡은 쪽도 **잣대가 조금씩 흔들립니다**. 처음에는 성능을 보다가 다음에는 코드 길이를, 그다음에는 설명이 그럴듯한 답을 고르는 식입니다. 그래서 중요한 작업일수록 **무엇을 보고 고를지 먼저 정해 두고**(요구사항을 지켰는지, 테스트를 통과했는지 정도면 충분합니다) 최종 판단은 사람이 합니다.

## 멀티는 공짜가 아니다 the cost table



| 이득                    | 비용                                                           |
|-----------------------|--------------------------------------------------------------|
| 서로 다른 갈래를 동시에 파볼 수 있음 | <b>토큰과 비용</b> 은 에이전트 수만큼 곱셈. 시간은 순차면 합, 병렬이면 가장 느린 하나에 맞춰집니다 |
| 다른 관점에서 한 번 더 봄       | 여러 에이전트가 같은 파일을 또 읽음 — 결과는 같은데 토큰만 두 배                       |
| 역할별로 전문화              | 무엇을 어디까지 하라고 했는지가 전달 과정에서 샘                                  |
| 작업 공간 분리              | 같은 파일 충돌 · 열여줘야 할 권한이 늘고 그만큼 뿔릴 통로도 늘어남                      |

강의에서 그대로 쓰는 문장

"멀티에이전트는 '많을수록 좋다'가 아닙니다. 하나의 에이전트나 워크플로보다 **측정 가능한 이점이 있을 때만** 겁니다."

☕ **파일 충돌은 이렇게 막습니다** — `git worktree add` 로 같은 저장소의 별도 작업 디렉터리를 파서, 에이전트마다 자기 사본에서 일하게 합니다. 한 공책을 여럿이 동시에 고치는 대신 각자 복사본에서 작업한 뒤 검증된 것만 합치는 셈입니다. 다만 마지막에 같은 줄을 고쳤다면 병합 충돌은 그대로 남습니다.

## 4 MCP · A2A — 연결 규격

에이전트가 도구·데이터·다른 에이전트와 만나는 표준 포트입니다.

### MCP · A2A 연결 규격들



**MCP**(Model Context Protocol) = 에이전트 ↔ 도구/데이터를 연결하는 표준 규격(host/client/server, tools/resources/prompts). **A2A** = 에이전트 ↔ 에이전트 통신 규격(Linux Foundation) ●.

☞ **쉽게 말하면** — MCP는 **USB-C 포트**입니다. 도구마다 전용 케이블을 만들지 않고, 규격에 맞춰 꽂으면 **MCP를 지원하는 앱이라면 같은 방식으로 연결**할 수 있습니다. 단, 아무 USB나 꽂으면 안 되듯 MCP 도구 설명에 명령을 심는 공격(tool poisoning)도 존재 — 신뢰할 수 있는 서버만.



MCP는 도구·데이터를, A2A는 다른 에이전트를 연결합니다.

**MCP는 손발을, A2A는 동료를 연결합니다.** MCP로 한 에이전트에 도구·데이터를 붙이고, A2A로 독립된 전문 에이전트에게 일을 맡겨 진행과 결과를 주고받습니다.

- **MCP** — 회의 비서 에이전트가 달력·문서·데이터베이스를 직접 조회
- **A2A** — 상담 에이전트가 청구 전문 에이전트에게 건을 위임
- **함께** — 출장 에이전트가 A2A로 전문가를 부르고, 각 전문가는 MCP로 예약·날씨 도구를 사용

⊗ **흔한 오해** — MCP 서버는 많이 붙일수록 좋습니다.

☑ **사실** — 많이 붙일수록 **엉뚱한 도구를 잘못 부르는 일**이 늘어납니다. 비슷한 이름의 도구가 여러 개면 에이전트가 헷갈립니다. 게다가 도구 설명에 명령을 숨겨 에이전트를 조종하는 공격도 실제로 있습니다. **지금 하는 일에 필요한 것만 켜 두고, 안 쓰는 것은 주기적으로 꺼 줍니다.** 삭제·외부 전송·결제처럼 되돌리기 어려운 일은 실행 전에 한 번 확인합니다. ●

## 5 RAG와 검색 — 그리고 대전환

"AI 코딩 = RAG"로 알려진 경우가 많습니다. 절반은 맞고, 절반은 2025년에 뒤집혔습니다.

### RAG Retrieval-Augmented Generation



질문이 오면 ① 관련 자료를 검색해서 ② 컨텍스트에 붙이고 ③ 모델이 그걸 근거로 답하게 하는 방식. 부품: **임베딩**(글→숫자 벡터), **벡터 DB**(비슷한 벡터 찾기), **칭킹**(문서 자르기), **리랭킹**(후보 재정렬).

문서는 통째로 넣지 않고 **조각**으로 잘라 두었다가, 질문과 가까운 조각만 골라 옵니다. 고르는 방법은 **키워드 검색**(단어가 그대로 일치)과 **벡터 검색**(뜻이 비슷)을 섞어 씁니다. 그렇게 가져온 조각을 근거로 답하고 **어느 문서에서 왔는지**까지 같이 보여 주는 것이 기본형입니다.

☞ **쉽게 말하면** — 오픈북 시험입니다. 모델의 머리(가중치)를 바꾸는 게 아니라, **시험장에 책을 갖고 들어 가게** 해주는 것. 환각(1장)의 1차 백신.

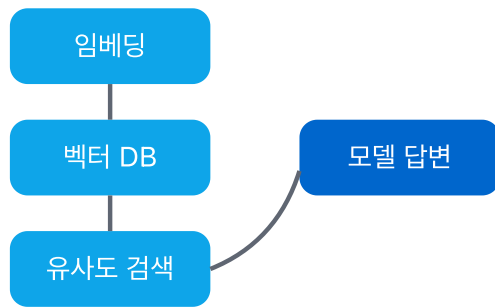


### 대전환: 벡터 → 에이전틱 검색 agentic search



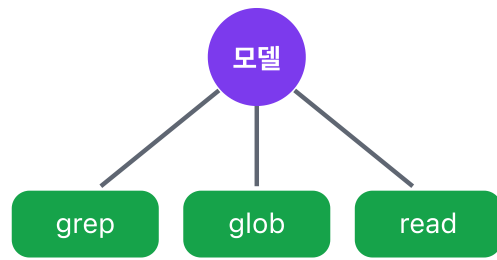
Claude Code는 초기에 벡터 RAG를 시험한 뒤 **grep**(정확한 문자열 검색) 기반 탐색을 선택했습니다 — 창업자 인터뷰 기준이며, 비교 근거는 "주로 체감 + 일부 내부 벤치"라고 직접 밝혔습니다 ●. 이후 Cursor·Cline 등이 유사한 방향으로 이동 ●. 에이전트가 검색 도구를 스스로 반복 사용하며 능동 탐색하는 방식이 **에이전틱 검색**입니다.

### 전통 RAG (전처리형)



인덱스 동기화 필요 · 문서에 강함

### 에이전틱 검색 (능동형)



도구를 반복 사용하며 스스로 탐색  
항상 현재 파일 상태 · 코드에 강함

그림 11 · 코드는 grep, 문서는 검색형 지식 — 용도가 다릅니다

⊗ **흔한 오해** — "그럼 RAG는 죽었네?"

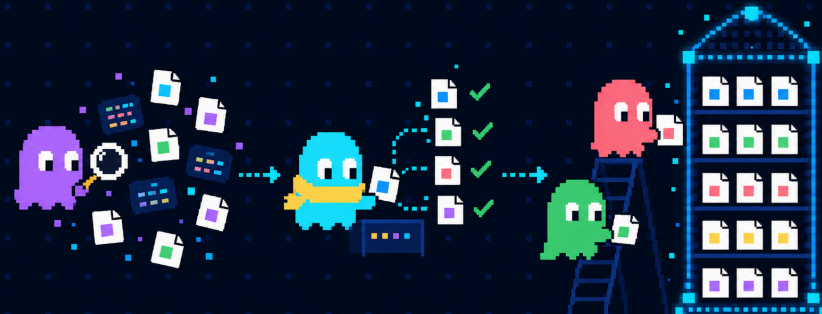
☑ **사실** — 코드 검색에서 퇴조했을 뿐, **문서·가이드·조직 내부 지식은 여전히 검색형(RAG·지식화)이 정답**입니다. 논문 "Is Grep All You Need?"(arXiv 2605.15184 ●)도 노이즈가 커지면 교차점이 생긴다고 답합니다. 요지는 "retrieval is really retrieval-plus-orchestration" — 검색은 하네스와 한 몸.

**에이전틱 RAG** = 언제, 무엇을, 다시 검색할지 에이전트가 스스로 정하고 중간 결과에 따라 전략을 바꾸는 형태입니다.

**GraphRAG** = 문서에서 **엔티티**(사람·제품·부서 같은 대상)와 **관계**를 뽑아 그래프로 엮고, 비슷한 것끼리 묶은 **공동체** 단위로도 찾을 수 있게 한 변형입니다. "A와 B의 연결고리는?"처럼 여러 문서를 건너뛰어야 하는 질문이나 "이 주제 전체를 요약해 줘"처럼 범위가 넓은 질문에 강합니다. 다만 **RAG의 다음 단계라거나 항상 더 낫다는 뜻은 아닙니다** — 그래프를 만들고 유지하는 비용이 따로 들기 때문에, 관계형 질문이 실제로 쌓였을 때 선택하는 도구입니다. ●

## 6 메모리와 LLM 위키 — 잊지 않는 에이전트

에이전트는 세션이 끝나면 전부 잊습니다. 이 망각과 싸우는 방법들.



검색은 지금 찾는 일, 위키는 검증한 지식을 다음 세션도 읽도록 차곡차곡 남기는 일.

### 지식 · 메모리 · 스킬 · 규칙 4구분 — 헛갈리면 여기로



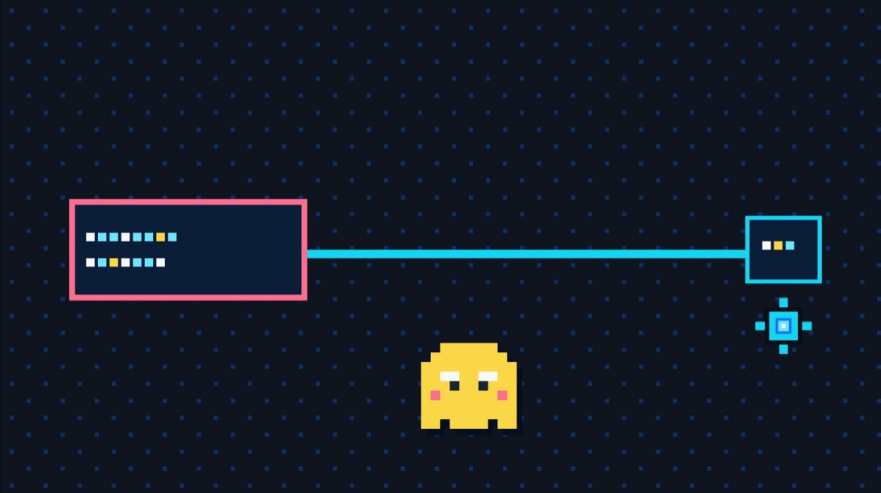
| 개념           | 정의                 | 수명                    | 예               |
|--------------|--------------------|-----------------------|-----------------|
| 지식 Knowledge | 지금 하는 작업에 찾아 쓰는 사실 | 작업 단위                 | 지식화된 .md, 검색 결과 |
| 메모리 Memory   | 작업이 끝나도 남는 정보      | 세션을 넘김                | 개인화 메모리, 사용자 선호 |
| 스킬 Skill     | 반복해서 쓰는 작업 절차      | 영구 · 옮겨 쓸 수 있음        | SKILL.md        |
| 규칙 Rule      | 허용·금지·승인 같은 행동 제약  | 영구 · 강제 <sup>*3</sup> | rules.md, 권한 설정 |

☕ **쉽게 말하면** — 지식은 **책상 위 자료**, 메모리는 **수첩**, 스킬은 **몸에 밴 레시피**, 규칙은 **사규**. 넣는 서랍이 다르면 꺼내는 방법도 다릅니다. 헛갈릴 때는 **얼마나 오래 남아야 하는가**로 나누면 됩니다.

### 컨텍스트 부패 · 압축 context rot · compaction



**Context rot** = 무관한 도구 출력과 낡은 중간상태가 쌓이며 모델이 **조용히** 정확도를 잃는 현상. 무서운 점: 예러가 안 납니다 ●. **Compaction** = 창이 차기 전에 대화를 요약해 새 창으로 이어붙이기 — 단, 무손실이 아니어서 압축 직후 "아까 정한 규칙"을 잊는 일이 흔합니다.



압축 · 가득 찬 창을 요약해 짧게 줄여 새 창으로 잇습니다. 줄이는 만큼 잃는 것도 있습니다.

실무 결론 한 줄

"중요한 결정은 대화가 아니라 **파일(저장소)**에 적어라." — OpenAI가 "repo가 단일 진실 원천"이라 말한 이유이자, 다음 용어(LLM 위키)의 존재 이유.

## LLM 위키 LLM wiki · 쓸수록 쌓이는 AI 지식창고



AI가 작업하면서 알게 된 내용과 **그렇게 정한 이유**를 마크다운 문서로 계속 정리해 두는 방식입니다. 다음 작업에서는 처음부터 다시 파악하지 않고, 정리해 둔 것을 읽은 뒤 그다음부터 이어서 일합니다. Karpathy가 2026-04 공개한 [LLM Wiki gist](#)의 패턴입니다. ●

☕ **쉽게 말하면** — 매번 기억을 잃는 AI에게 **지난 작업 기록을 만들어 주는 것**입니다.

### RAG

외부 문서를  
그때그때 검색

"찾아 읽는다"

### 에이전트 메모리

대화·경험에서  
자동 추출·저장

"겪으며 기억한다"

### LLM 위키

사람이 읽을 수 있는  
md 페이지로 정제·누적

"기록하며 복리로 쌓는다"

셋은 경쟁이 아니라 조합 — 코드는 grep, 문서는 RAG, 결정은 위키, 선호는 메모리

그림 12 · 망각과 싸우는 세 무기의 역할 분담

"쌓인다"는 말의 뜻 — 질문할 때마다 처음부터 다시 조사하지 않는다는 뜻입니다. 어제 정리해 둔 것 위에 오늘 알게 된 것을 붙이니까, 쓸수록 다시 조사할 양이 줄어듭니다. 저절로 붙어나는 것이 아니라 **한 번 정리한 것을 다시 쓰는 것**입니다.

사람이 안 쓰고 자동으로 남기는 방식도 있습니다. 예를 들어 `claude-mem` 은 작업 중에 오간 대화와 도구 실행 결과를 자동으로 기록해 두었다가, 통째로 쌓지 않고 **핵심만 요약해** 다음 작업이 시작될 때 다시 넣어 줍니다. 손으로 쓰는 `CLAUDE.md` 와 반대쪽 방식이고, 둘을 같이 쓰기도 합니다. ●

학계에서는 스스로 메모리를 연결·진화시키는 **A-MEM**이나 계층형 메모리를 쓰는 **Letta**처럼 더 복잡한 구조도 연구합니다. 다만 코딩 실무의 결론은 여전히 **중요한 것을 파일과 저장소에 남기는 방식이 가장 단순하고 확실하다**는 것입니다.

☞ **핵심** — 대화는 사라져도 **위키에 정리된 지식은 다음 작업에 그대로 이어집니다.**

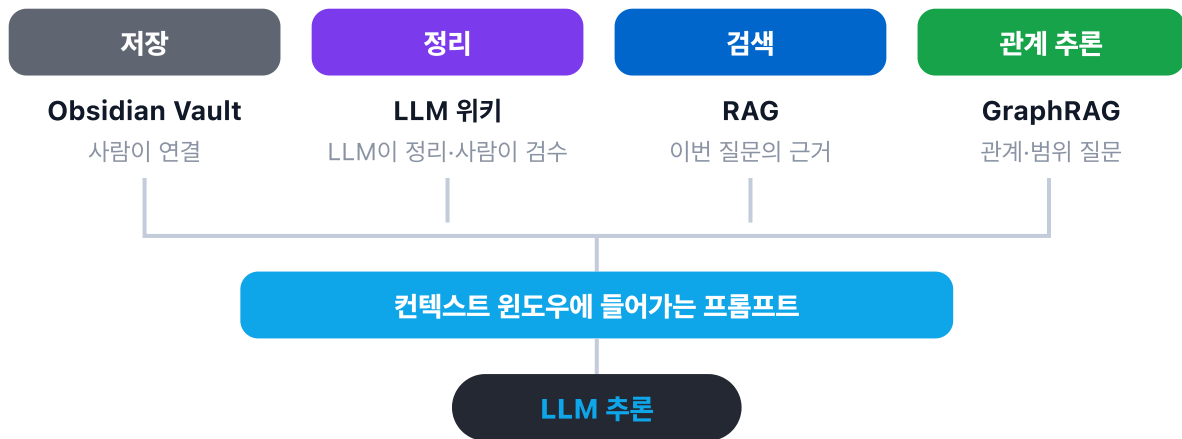
**지식 시스템 4가지 역할** store · curate · retrieve · relate



Obsidian Vault, LLM 위키, RAG, GraphRAG는 자주 함께 언급되지만 **같은 것을 두고 경쟁하지 않습니다.** 맡는 역할이 다릅니다.

| 역할                          | 무엇인가                                 | 언제 쓰나                     |
|-----------------------------|--------------------------------------|---------------------------|
| <b>저장</b><br>Obsidian Vault | 사람이 위키링크로 연결하는 마크다운 지식 저장소           | 원본과 연결 관계를 사람이 직접 남겨 둘 때  |
| <b>정리</b><br>LLM 위키         | 대화과 작업 기록을 LLM이 정리하고 사람이 검수하는 문서 체계  | 같은 결정을 두 번 설명하고 싶지 않을 때   |
| <b>검색</b><br>RAG            | 질문에 가까운 근거를 찾아 이번 요청의 컨텍스트에 넣어 주는 방식 | 답할 때마다 필요한 자료가 달라질 때      |
| <b>관계 추론</b><br>GraphRAG    | 엔티티와 관계를 그래프로 엮어 관계형·광범위 질문에 답하는 변형  | 여러 문서를 건너뛰는 질문이 실제로 쌓였을 때 |





넷을 다 갖춰야 완성되는 것이 아니라, 지금 필요한 역할만 조합하면 됩니다

그림 13 · 저장 · 정리 · 검색 · 관계 추론 — 경쟁 제품이 아니라 서로 다른 역할

☕ **쉽게 말하면** — 네 가지는 **경쟁 제품이나 고정된 발전 순서가 아니라, 저장·정리·검색·관계 추론을 담당하는 서로 다른 역할**입니다. 저장은 원본을 남기고, 정리는 다시 쓸 수 있게 다듬고, 검색은 지금 필요한 근거를 꺼내 오고, 관계 추론은 흩어진 것들을 이어 줍니다.

## 어떻게 조합할까 필요한 역할만



순서를 정해 두면 결정이 쉬워집니다.

- 파일과 마크다운을 기준 저장소로 둡니다. 도구가 바뀌어도 남는 것은 파일입니다.
- 필요한 문서는 RAG로 찾습니다. 매번 통째로 넣지 않고 질문에 가까운 조각만 꺼내 옵니다.
- 반복해서 쓰는 결정과 지식은 LLM 위키로 정리합니다. 두 번 설명하게 되는 것이 정리 대상입니다.
- 관계형 질문이 실제로 쌓였을 때 GraphRAG를 선택적으로 도입합니다. 먼저 만들어 두는 것이 아닙니다.

⊗ **흔한 오해** — 네 가지를 모두 구축해야 지식 시스템이 완성됩니다.

☑ **사실** — GraphRAG까지 모두 구축해야 완성되는 것은 아닙니다. 현재 필요한 역할만 조합하면 됩니다. 그래프를 만들고 최신 상태로 유지하는 비용은 계속 들기 때문에, 답해야 할 질문의 성격이 먼저입니다.

☕ **주의** — RAG가 만능은 아닙니다. 질문과 상관없는 조각까지 끌어와 오히려 답을 흐리는 일이 잦고, 문서를 쪼개 넣고 계속 최신으로 맞추는 데 **품과 비용이 꾸준히 듭니다**. 파일 몇 개를 그냥 읽히면 될 일에 검색을 붙이는 것이 가장 흔한 낭비입니다. **먼저 그냥 읽혀 보고, 그래도 안 될 때 검색을 붙입니다**.

한 줄로

"파일에 남기고, 필요할 때 찾아 쓰고, 반복되면 정리하고, 관계가 문제일 때만 그래프를 엮습니다."

## 7 하네스(하네스 엔지니어링)

모델은 제안만 합니다. 실제로 도구를 쓰고, 결과를 확인하고, 못 하게 막는 것은 모두 모델 바깥입니다. 하네스 하네스 하네스 하네스 하네스... 대체 뭔데?

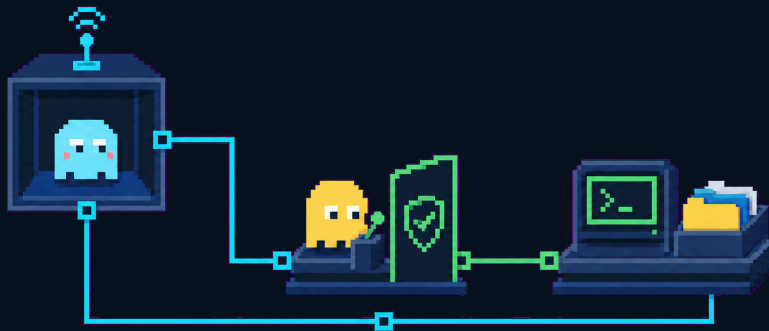
**하네스** harness — Agent ≈ Model + Harness



하네스는 모델을 실제 작업으로 연결하고, 도구 실행·상태·권한·검증을 관리하는 실행 계층입니다. 그래서 **Agent ≈ Model + Harness**라고 설명합니다. 한 줄로 줄이면, 하네스는 아무것도 직접 할 수 없는 모델에 손과 발을 달아 주고, 정해 둔 규칙 안에서만 움직이게 하는 실행 틀입니다.

발표용 한 줄

"좋은 모델을 고르는 것보다, 모델을 끝까지 일하게 만드는 하네스를 설계하는 일이 더 어렵습니다."  
— 인터뷰의 취지를 재구성한 문장입니다. 직접 인용은 아닙니다. ●



유령 비유 · 모델의 판단이 승인 게이트를 거쳐 실행되고, 결과가 다시 모델로 돌아옵니다.

### 하네스 — 모델 바깥의 실행 환경 전부

도구 실행 · 권한 · 결과 확인 · 재시도 · 종료 판단

지침 파일(AGENTS.md · CLAUDE.md) · 훅 · 스킴 · 서브에이전트

도구 실행

검증

권한·격리

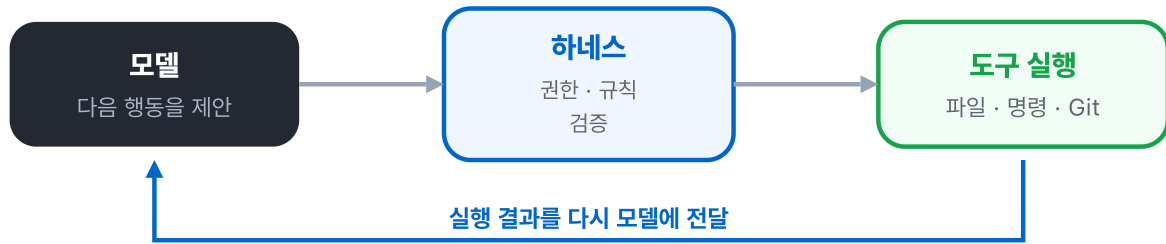
기록·복구

**모델**

판단하고 다음 행동을 제안

모델은 제안하고, 하네스가 실제로 실행합니다

그림 14 · 모델을 감싼 하네스 — 모델 빼고 나머지가 전부 하네스입니다



허용된 도구·권한·규칙 안에서만 움직이게 하는 것이 하네스의 역할입니다

그림 15 · 모델은 제안, 하네스는 실행 — 실행 결과는 다시 모델에 돌아가 다음 판단의 근거가 됩니다

모델은 상황을 읽고 **다음에 무엇을 할지 제안**할 뿐이고, 파일을 열고 명령을 실행하는 것은 하네스입니다. 지침 파일 ( AGENTS.md · CLAUDE.md ), 혹은, 스킴, 서브에이전트도 모두 하네스에 속합니다. 자료에 따라 이 부분만 따로 떼어 **스캐폴드**라고 부르기도 하는데, 포함 관계가 자료마다 뒤집혀서 여기서는 **하네스 하나로 부릅니다**.

커뮤니티 도구는 기존 하네스 위에 스킴·서브에이전트·워크플로를 얹는 확장 계층입니다. 일부는 자체 실행 루프와 상태 관리까지 맡아 바깥쪽 하네스 역할을 합니다.

| 역할 | 하는 일                                                                                                                                    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------|
| 실행 | 도구를 쓰고, 작업 순서와 맥락을 관리합니다.                                                                                                               |
| 검증 | 테스트·빌드·린트로 결과를 확인하고, 실패하면 다시 시도하거나 이전 상태로 되돌립니다.                                                                                        |
| 통제 | 권한·비용·작업 범위와 <b>어떤 데이터에 닿을 수 있는지</b> 를 제한하고, 무엇을 어디까지 했는지 <b>지켜보며</b> 기록을 남깁니다. 파일·네트워크 접근을 격리하는 샌드박스가 4장에서 본 tool poisoning의 방어선입니다. ● |

☕ **핵심** — AI가 "완료했습니다"라고 말하는지가 아니라 **실제 실행 결과**를 믿습니다. Tool Call만 연결했다고 하네스가 되는 것은 아니고, **실행·검증·통제가 함께** 있어야 합니다.

한 줄 정의

하네스는 불안정한 AI 작업을 관리 가능한 실행 과정으로 바꿉니다.

## 하네스 엔지니어링 harness engineering



하네스 엔지니어링은 가드레일, 문서화된 지침, 피드백 루프를 활용해 AI 에이전트의 외부 작업 환경을 구축하고 제어하는 기술입니다.

- **프롬프트 엔지니어링** — AI에게 무엇을 시킬지 지시합니다.
- **컨텍스트 엔지니어링** — 추론할 때 무엇을 보여줄지 구성합니다.

- 하네스 엔지니어링 — 에이전트가 어떻게 일할지 환경·권한·검증을 설계합니다.

 **쉽게 말하면** — 프롬프트는 지시문, 컨텍스트는 참고자료, 하네스는 작업장·안전장치·검증 절차입니다.

⊗ **흔한 오해** — 좋은 프롬프트만 쓰면 에이전트가 안전하게 끝까지 일합니다.

✓ **사실** — 실제 작업은 권한·도구·테스트·로그 같은 시스템 통제가 결정합니다.

## 에이전틱 코딩의 기본 흐름

계획 → 개발 → 검토 → 수정 반복



먼저 **무엇을 만들지와 어디까지 되면 끝인지**를 간단히 정합니다. 그다음 AI가 개발하고, 테스트와 리뷰로 확인한 뒤 문제가 있으면 다시 고칩니다. 도구가 무엇이든 실제로 도는 모양은 거의 같고, 이 책자를 만든 과정도 그대로였습니다.

### 한 줄로

요즘 에이전틱 코딩은 **계획하고, 개발하고, 검토한 뒤 통과할 때까지 다시 고치는** 방식입니다.

"만들기 전에 **이러면 된 것이다**를 먼저 적어 둔다" — 개발 쪽에서 오래 쓰던 방식입니다. 부르는 이름은 있지만 몰라도 상관 없습니다. 중요한 건 끝났다는 **판단을 사람 말이 아니라 미리 정해 둔 조건에 맡긴다**는 점입니다. 에이전트와 일할 때 이게 특히 잘 듣는 이유는, 에이전트가 "다 했습니다"라고 말하는 것과 실제로 된 것이 자주 다르기 때문입니다.

| 단계        | 하는 일                        | 이 책자를 만들 때 실제로                              |
|-----------|-----------------------------|---------------------------------------------|
| 1 · 합격 기준 | 무엇이 되면 끝인지 <b>먼저</b> 적습니다   | "A4에서 페이지가 잘리지 않을 것 · 가로 스크롤 0 · 콘솔 에러 0"   |
| 2 · 구현    | 에이전트가 만들거나 고칩니다             | 카드·그림·인쇄 CSS 수정                             |
| 3 · 검증    | 사람 말이 아니라 <b>기계가 확인</b> 합니다 | 실제 브라우저 375·768·1280px 측정, A4 PDF 28쪽 전수 점검 |
| 4 · 판단    | 기계가 못 정하는 것만 사람이 정합니다       | 그림을 바꿀지, 문구 톤을 어디까지 낮출지                     |
| 5 · 기록    | 고른 이유와 <b>버린 이유</b> 를 남깁니다  | 커밋에 채택·기각 사유와 다음 사람에게 주는 경고를 함께             |

☞ **쉽게 말하면** — 완료는 에이전트가 선언하는 것이 아니라 검증이 증명합니다. 그래서 3번을 사람이 눈으로 훑는 대신 명령 하나로 돌 수 있게 만들어 두는 것이 가장 크게 남습니다. 5번은 다음에 같은 고민을 반복하지 않기 위한 것이고, 특히 **버린 이유**가 고른 이유보다 오래 쓸모 있습니다.

⊗ **흔한 오해** — 테스트가 초록불이면 검증은 끝난 것이다.

☑ **사실** — 코드와 테스트를 같은 에이전트가 함께 만들면 **자기 구현에 맞춘 테스트**가 붙기 쉽습니다. 늘 100원을 돌려주는 계산기에 100원을 기대하는 테스트를 달면 초록불은 켜지지만 아무것도 검증하지 못합니다. 기대값은 사람이 먼저 정하고, 경계값과 반례를 넣고, 테스트 코드도 리뷰합니다. ●

# 8 루프 엔지니어링

하네스가 일할 환경을 만드는 기술이라면, 루프는 그 환경에 일감을 계속 넣어 주는 기술입니다.

## 루프 엔지니어링 loop engineering



루프 엔지니어링은 AI가 목표를 끝낼 때까지 실행·검증·수정을 자동으로 반복하게 만드는 설계입니다.

한 바퀴

목표 설정 → AI 실행 → 테스트 → 실패하면 수정 → 다시 실행 → 성공하면 종료

사람이 매번 다음 지시를 내리는 대신, 시스템이 다음 작업과 완료 여부를 관리합니다.

| 구분    | 역할                              |
|-------|---------------------------------|
| 실행 관리 | 언제 시작하고 무엇을 다시 할지 정합니다.         |
| 작업 환경 | 코드·도구·규칙을 연결하고 서로 충돌하지 않게 합니다.  |
| 협업·기록 | 여러 에이전트가 역할을 나누고, 결과와 실패를 남깁니다. |

☕ **핵심** — AI의 "완료했습니다"라는 말이 아니라 **테스트·빌드·린트 결과로 완료를 판단합니다.**

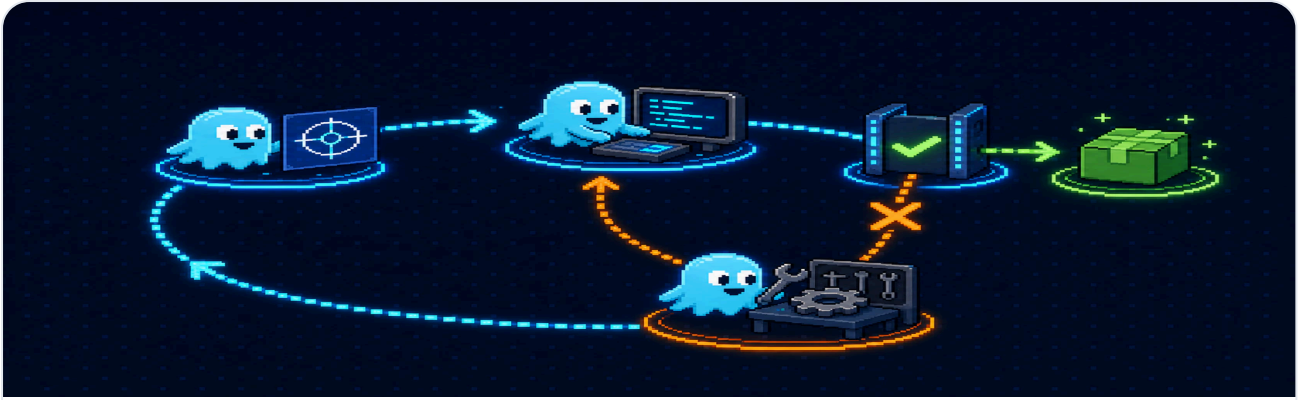
⊗ **흔한 오해** — 루프를 만들면 사람이 필요 없습니다.

☑ **사실** — 사람을 없애는 기술이 아니라 **사람이 반복해서 지시하고 확인하는 일을 줄이는** 기술입니다. 최종 목표와 제품 반영 여부는 사람이 판단합니다.

| 세 용어       | 쉽게 말하면               |
|------------|----------------------|
| 프롬프트 엔지니어링 | 무엇을 시킬지 설계           |
| 하네스 엔지니어링  | 어떤 환경에서 일하게 할지 설계    |
| 루프 엔지니어링   | 끝날 때까지 어떻게 반복시킬지 설계  |
| 그래프 엔지니어링  | 여러 루프를 어떤 갈래로 이을지 설계 |

한 문장으로

루프 엔지니어링은 AI가 테스트를 통과할 때까지 실행과 수정을 스스로 반복하게 만드는 기술입니다.



### 실패하면 고쳐서 다시

통과할 때만 완료 상자로 나가는 반복 구조

## 프롬프트도 루프가 씁니다 automatic prompt optimization · 자동 프롬프트 최적화



여기까지의 루프는 **코드**를 고쳤습니다. 같은 루프를 **프롬프트 자체**에 걸 수도 있습니다. 사람이 문구를 손으로 다듬는 대신, 채점하고 고쳐 쓰는 일을 시스템에 맡기는 것입니다.

### 한 바퀴

프롬프트 후보로 실행 → 채점 → 틀린 이유를 근거로 다시 씀 → 반복 → **가장 점수 높은 문구를 채택**

모양이 앞의 루프와 같습니다. **고치는 대상만 코드에서 프롬프트로 바뀌었을 뿐입니다.**

| 도구          | 하는 일                                                               |
|-------------|--------------------------------------------------------------------|
| <b>DSPy</b> | 문구를 직접 쓰지 않고 <b>입력과 출력이 무엇인지만 선언</b> 하면, 채점 기준에 맞게 문구와 예시를 맞춰 줍니다. |
| <b>OPRO</b> | 모델이 <b>스스로 새 문구 후보를 내고</b> 점수를 매겨 더 나은 쪽으로 나아갑니다.                  |
| <b>GEPA</b> | 실패한 기록을 <b>말로 되짚어</b> 문구를 고쳐 씁니다. DSPy 안에서 바로 쓸 수 있습니다.            |

☕ **전제** — 이게 돌아가려면 **채점 기준**이 있어야 합니다. 무엇이 맞고 틀린지 기계가 셀 수 없으면 루프가 돌지 않습니다. 그래서 먼저 하는 일은 프롬프트를 잘 쓰는 것이 아니라 **채점할 문제집을 만드는 것**입니다.

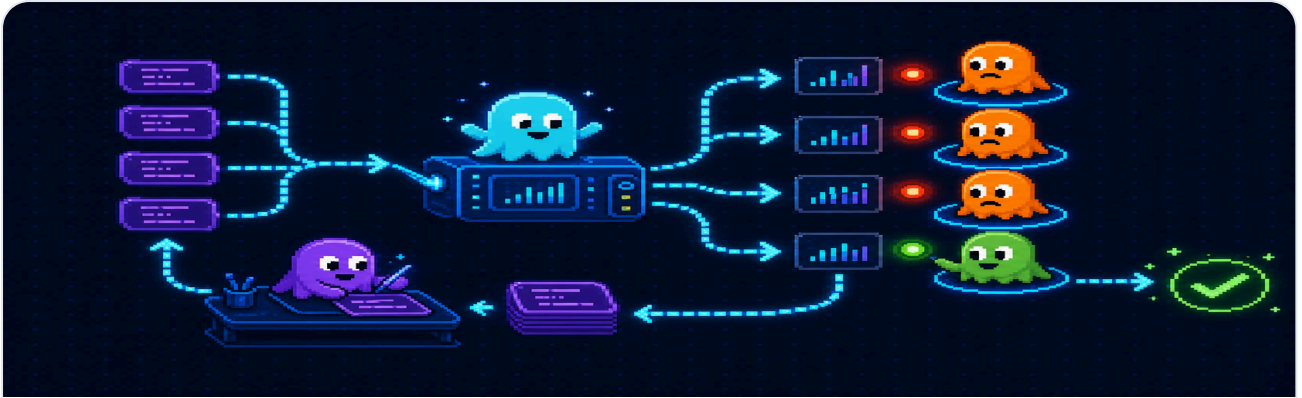
⊗ **흔한 오해** — 잘 듣는 마법의 문구를 찾아 외워 두면 됩니다.

☑ **사실** — 널리 알려진 "Take a deep breath and work on this problem step by step"도 사람이 지은 것이 아니라 이 루프가 찾아낸 문구입니다. 다만 **특정 모델(PaLM 2)**과 **특정 시험(GSM8K)**에서 **나온 결과**라 모델이 바뀌면 좋은 문구도 바뀝니다. 남는 것은 문구가 아니라 **문구를 찾아내는 루프**입니다.

### 달라지는 것

프롬프트를 쓰는 사람에서, 프롬프트를 만들어 내는 **시스템을 설계하는 사람**으로 옮겨 갑니다.



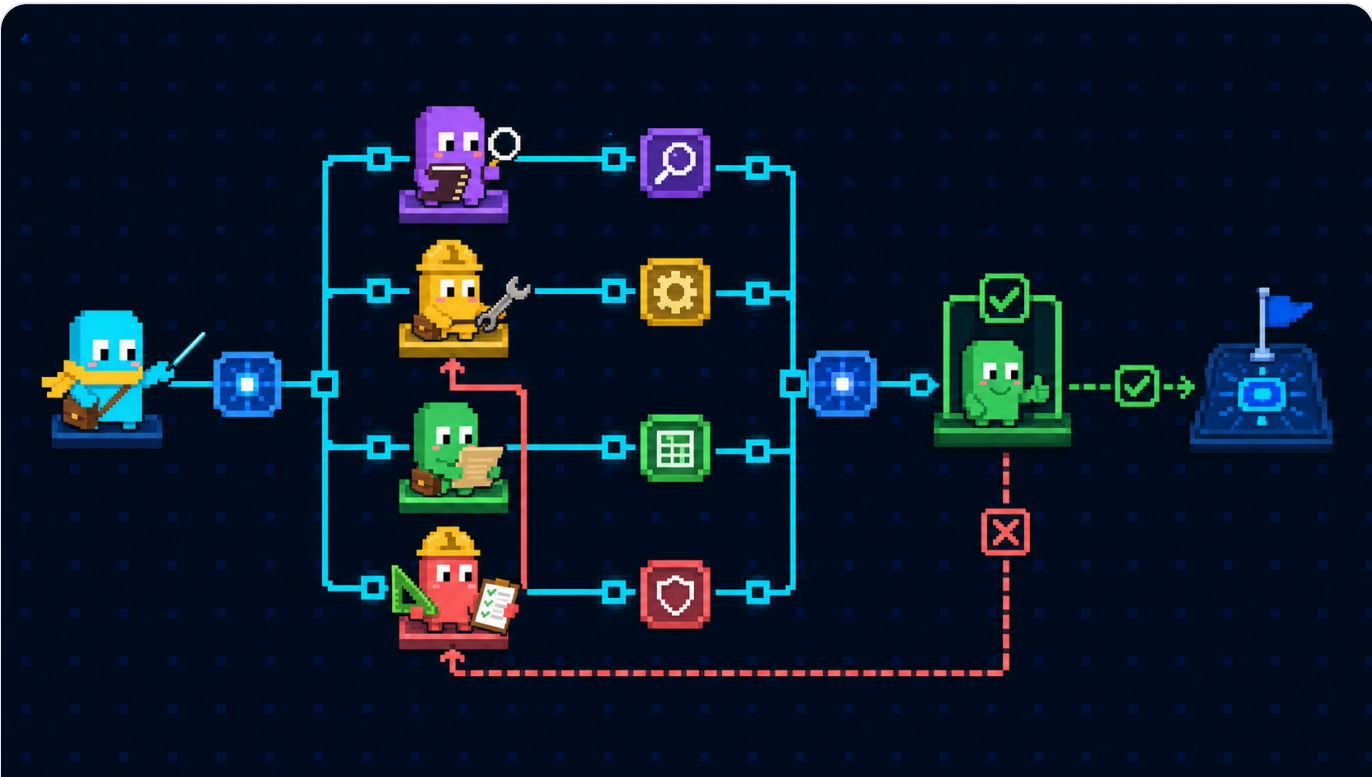


후보 생성 → 평가 → 재작성 → 선택

문구를 손으로 고치지 않고 시스템이 반복합니다

# 9 그래프 엔지니어링

루프 엔지니어링이 AI 하나를 끝까지 일하게 만드는 것이라면, 그래프 엔지니어링은 여러 AI와 도구가 어떤 순서와 관계로 일할지를 설계하는 것입니다.



그래프 시스템 맵  
노드·엣지·상태가 연결된 전체 실행 관계

## 그래프 엔지니어링 graph engineering 😊

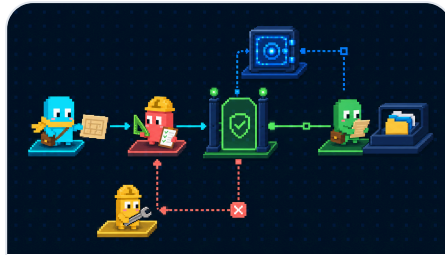
에이전트·도구·함수·사람의 승인을 **노드**로 놓고, 그 사이를 잇는 순서와 갈래를 설계하는 일입니다. 쉽게 말하면 AI 업무용 **순서도**, 또는 **지하철 노선도**를 그리는 것입니다.

| 세 요소    | 무엇에 해당하나                                     |
|---------|----------------------------------------------|
| 노드 · 역  | 실제로 일하는 자리. 에이전트, 모델 호출, 함수, 테스트, 도구, 사람 승인. |
| 엣지 · 선로 | 다음 역으로 가는 길. 순차, 조건 분기, 병렬, 실패 시 되돌아가기.      |
| 상태 · 승객 | 역 사이를 옮겨 다니는 작업 데이터. 요청, 코드, 중간 결과, 승인 여부.   |



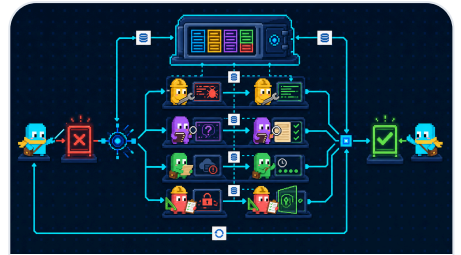
#### 병렬 분기와 합류

전문 작업을 나누고 결과를 다시 모읍니다.



#### 사람 승인 체크포인트

위험한 실행은 사람 확인 뒤에 통과시킵니다.



#### 실패 라우팅과 상태

상태와 실패 원인에 맞춰 다음 경로를 고릅니다.

⊗ **흔한 오해** — 최근에 새로 발명된 기술입니다.

☑ **사실** — 워크플로우, 상태 머신, DAG, 멀티에이전트 오케스트레이션은 원래 있던 개념이고 그래프 기반 프레임워크도 2024년부터 쓰였습니다. 새 구조가 생긴 것이 아니라 **그 설계 영역에 이름이 붙은 것에** 가깝습니다.

☕ **핵심** — **루프도 그래프의 한 종류입니다.** 자기 자신으로 돌아오는 노드 하나짜리 그래프라, 그래프는 루프를 대체하지 않고 **여러 루프를 있습니다.** 전문 영역이 갈리거나, 병렬로 합쳐야 하거나, 실패 원인마다 갈 곳이 다르거나, 사람 승인이 낄 때만 넓히고 **그 밖에는 루프 하나로 충분합니다.**

#### 노드를 늘리기 전에

노드가 늘면 호출량도 늘니다. Anthropic은 자사 리서치 시스템에서 에이전트가 일반 대화의 약 4배, 멀티 에이전트가 약 15배의 토큰을 썼다고 밝혔습니다. 새 노드의 값어치가 비용보다 클 때만 나눕니다.

#### 헛갈리기 쉬움

여기서 말하는 그래프는 **실행 경로를 그리는 컨트롤 그래프**입니다. 문서 속 관계를 저장해 찾는 **GraphRAG의 지식 그래프**와는 이름만 같고 푸는 문제가 다릅니다.

## 🔍 근거 표시 안내

이 자료는 가능한 한 공식 문서와 1차 출처를 우선해 작성했습니다. 인터뷰, 프리프린트, 커뮤니티에서 논의되는 내용은 확인된 사실과 구분해 표시했습니다.

### ● 확인된 근거

공식 문서, 논문, 공개된 수치처럼 출처를 직접 확인한 내용입니다.

### 해석·정리

여러 자료를 바탕으로 이해하기 쉽게 요약하거나 재구성한 내용입니다.

### ● 연구·참고 단계

인터뷰, 프리프린트, 초기 제안처럼 아직 정의나 평가가 달라질 수 있는 내용입니다.

가능한 한 객관적인 자료를 보여 드리기 위해 출처와 표현의 강도를 구분했습니다. 초록 표시는 **원문을 직접 확인했다**는 뜻이지, 그 내용이 업계에서 확정된 표준이라는 뜻은 아닙니다. 다만 빠르게 변하는 분야이므로 일부 용어와 평가는 달라질 수 있습니다.

## 추신 · 참조 — 본문의 \*1~\*6

쉽게 읽히도록 줄여 쓴 표현들의 정확한 범위입니다. 본문과 함께 읽어 주세요.

- \*1 그라운드링의 "최대한 많이"는 정확히는 **관련 있고 믿을 수 있는 근거를 필요한 만큼 골라** 주는 것입니다. 많이 넣을수록 좋아지는 것은 아닙니다 — 창이 차면 정작 필요한 것이 묻힙니다(1장 컨텍스트 윈도우·6장 RAG 주의와 같은 이유).
- \*2 컨텍스트 윈도우는 입력 전용 공간이 아니라 **출력과 추론 모델의 생각 토큰까지 함께 쓰는 전체 공간**입니다. "실효 200~400K"도 공통 기준이 아니라 모델·작업에 따라 달라지는 참고치입니다.
- \*3 문서(rules.md)에 적은 규칙은 행동을 **안내**하고, 실제 **강제**는 권한 설정·승인·샌드박스 같은 시스템 통제가 담당합니다 — 7장 하네스의 통제와 같은 이야기입니다.
- \*4 "같은 구조"는 **가중치가 바뀌지 않는다**는 뜻입니다. 구분하면 — 검색해서 관련 문서를 골라 넣으면 RAG, 정해 둔 파일을 그대로 붙이면 컨텍스트 주입이고, LLM 위키는 후자에 가깝습니다. 어느 쪽이든 문서를 넣는 것만으로 모델이 학습되지는 않습니다.
- \*5 "지식화"는 문서를 파싱·정리해 검색 가능한 상태로 준비하는 과정을 묶어 부르는 **실무 표현**입니다. 영문 병기 knowledgeization은 표준 용어가 아니며, 영어권에서는 document ingestion·knowledge-base preparation 같은 표현을 씁니다.
- \*6 Chroma 수치는 특정 과제·조건에서의 결과라 모든 모델·작업에 그대로 적용되는 기준은 아닙니다. 원문은 출처 목록의 **Context Rot — Chroma Research**에 연결해 두었습니다.

## 출처 · 더 읽을거리

[Harness engineering: leveraging Codex in an agent-first world — OpenAI](#)  
[The Anatomy of an Agent Harness \(Agent = Model + Harness\) — LangChain](#)  
[Improving Deep Agents with Harness Engineering \(Top 30→Top 5\) — LangChain](#)  
[My AI Adoption Journey — Mitchell Hashimoto](#)  
[Effective harnesses for long-running agents — Anthropic](#)  
[Extreme Harness Engineering — Ryan Lopopolo 인터뷰, Latent Space](#)  
[What makes a harness a harness \(T1–T4\) — arXiv 프리프린트](#)  
[Harness engineering for coding agent users — martinfowler.com](#)  
[Harness, Scaffold, and the AI Agent Terms Worth Getting Right — HuggingFace](#)  
[What is an AI Agent Harness? — Databricks](#)

[Anthropic 2026 Agentic Coding Trends Report](#)  
[The Code Agent Orchestra — Addy Osmani](#)  
[Loop Engineering — Addy Osmani \(Substack, 2026-06\)](#)  
[Context Rot: How Increasing Input Tokens Impacts LLM Performance — Chroma Research](#)  
[Large Language Models as Optimizers \(OPRO\) — arXiv](#)  
[GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning — arXiv](#)  
[DSPy — Stanford NLP](#)  
[Test Driven Development — martinowler.com](#)  
[Best practices for using AI in VS Code — Microsoft, VS Code Docs](#)  
[Writing tests with GitHub Copilot — GitHub Docs](#)  
[Spec Kit — Spec-Driven Development \(GitHub\)](#)  
[Tracing — OpenAI Agents SDK](#)  
[Agentic Engineering Patterns — Simon Willison](#)  
[Claude Code sandboxing — Anthropic](#)  
[Multi-Agent Orchestration: 5 Patterns — Digital Applied](#)  
[A-MEM: Agentic Memory for LLM Agents — arXiv](#)  
[RAG vs Agent Memory vs LLM Wiki — dev.to](#)  
[LLM Wiki gist — Andrej Karpathy, 2026-04](#)  
[LangGraph — 노드.엣지.상태 기반 에이전트 그래프 \(LangChain Docs\)](#)  
[How we built our multi-agent research system — Anthropic \(토큰 4배.15배 수치\)](#)



저작권·제작 크레딧

© 2026 김지원 · 출처를 밝히면 비영리로 자유롭게 쓰셔도 됩니다

CC BY-NC 4.0 · 2026

직접 만들었습니다

구성·문안·도식·자체 캐릭터 콘셉트·편집 디자인을 직접 기획하고 만들었습니다. 직접 만든 삽화에 남의 캐릭터나 로고 원본을 가져다 쓰지 않았습니다.

AI를 도구로 썼습니다

일부 삽화는 OpenAI 이미지 생성 도구를 보조로 사용했습니다. 콘셉트, 프롬프트, 결과 선별, 편집과 배치는 직접 했습니다.

출처는 다 밝혔습니다

외부 문헌은 참고해 직접 다시 썼고, 원문을 그대로 옮긴 부분은 인용으로 표시했습니다. 저자와 원문은 위 출처 목록에 있습니다.

제품 화면과 상표

일부 제품 화면은 기능 설명을 위해 직접 캡처해 실었습니다. 화면·UI·상표의 권리는 각 권리자에게 있고, 이 자료는 그들과 제휴·후원 관계가 없습니다.

자세한 이용 조건은 [LICENSE](#)에 있습니다.